

Sethu

The bridge for API breaking changes.

Powered by IBM Bob



One API change can break an app

Random search returned an object. Now it returns an array.

🔍 Read file `src/lib.rs` (218-245) >



🔗 Applied diff to `src/lib.rs` >

Now update the tests. The changes needed are:

1. `fixture()` for random: bare array instead of wrapped object — but `smart` and `metadata` still use the wrapped format. Split into two helpers.
2. Remove `page` assertions from random tests.
3. Remove `next_page()` assertions.
4. Update call sites from `search_random(1)` to `search_random()`.

🔗 Applied diff to `tests/search_picker.rs` >

Now update `src/main.rs` — the call to `picker.more()` doesn't change its signature, but let's verify nothing else in `main.rs` needs updating.

🔍 Read file `src/main.rs` >

`src/main.rs` is clean — uses `picker.more(&client)` which hasn't changed signature. Now compile and run tests.

📄 Ran `cargo test 2>&1` 📄 >

All 8 tests pass. Now proceeding to Steps 6 and 7.

Vimanam finds what changed

It compares two API versions. Sethu tracks twenty-seven Immich changes into the app.

Step 1 — sethu init OLD NEW

Step 1 — init

Ran sethu init /home/nryn/work/sethu/tests/fixtures/immich/old.json /home/nryn/work/sethu/tests/fi...

Step 2 — sethu changes, split into groups

Init succeeded: 46 changes across pair ff2d4e2a7c35-a5c98c5cf35f, attempt b9d4eb22. Baseline commit 3074c1f0.

Step 2 — changes

Ran sethu changes --json 2>&1

Step 3 — sethu context --prepare per group, launch tracers

27 required changes (all breaking), grouped into 3:

Group	Origin	Changes	Operations
1	RandomSearchDto component	1	POST /search/random
2	SystemConfigImageDto component	22	GET/PUT /system-config, GET /system-config/defaults
3	POST /search/random 200 response type change	4	POST /search/random

Find the affected code

Step 3 — sethu context --prepare per group, launch tracers

27 required changes (all breaking), grouped into 3:

Group	Origin	Changes	Operations
1	RandomSearchDto component	1	POST /search/random
2	SystemConfigImageDto component	22	GET/PUT /system-config, GET /system-config/defaults
3	POST /search/random 200 response type change	4	POST /search/random

Step 3 — context --prepare per group, then tracers

Preparing all three groups in sequence.

Ran 2 commands >

--prepare <GROUP> takes a group number but requires an <ID> positional argument too — the ID is the first change ID of the group. Preparing each group using the first change ID of each.

Ran 3 commands >

All three groups are prepared. Now launching one explore tracer per group in parallel. The base context path is .sethu/migrations/b9d4eb223af677a295263ffc36b8b930372f3783f70caac5dce13e5de848c1f7/context/. Each tracer reads its prepared context and application source, returning evidence references and uncertainties only — no commands, no writes.

You are a read-only tracer for a migration. Your job is to read the prepared context files and the application source, then ...

Done · 25 tools · 22.2k · @0.181 · 44s

You are a read-only tracer for a migration. Your job is to read the prepared context files and the application source, then ...

Done · 31 tools · 30.9k · @0.259 · 1m 8s

You are a read-only tracer for a migration. Your job is to read the prepared context files and the application source, then ...

Done · 48 tools · 42.9k · @0.559 · 1m 39s

All three tracers have returned. Now merging their findings and assigning per-ID dispositions before editing.

Bob traces the changes into the picker.

Repair the one broken path

```
Step 5 — Merge findings, assign per-ID dispositions, edit co...

Step 5 — Merge & dispositions

Reading the application source to confirm tracer findings before editing.

Explored 2 files : 1ms >

I now have full clarity on all 27 change IDs. Here is the merged disposition plan before I write a single line of code:

Group 2 (all 22 IDs — SystemConfigImageDto): This application uses only POST /search/random, /search/smart, and /search/metadata. It never calls GET /system-config, GET /system-config/defaults, or PUT /system-config. Outcome: no_usage_found for all 22.

Group 1 (1 ID — RandomSearchDto, page removal from request): The app sends PageBody { page } to POST /search/random. The field is now removed from the schema. The PageBody struct, search_random() signature, Picker::next_page counter, and Picker::more() logic must all be updated. The tests asserting "page":1 and "page":2 in the body must also be updated.

Group 3 (4 IDs — response type object->array): POST /search/random now returns Vec<AssetResponseDto> (bare array), not SearchResponseDto (object with .assets.items). The single post() method currently parses all three routes identically to SearchResponse. Only the random route changes. The repair must:

1. Give search_random its own parser that accepts a bare Vec<Asset> and normalises it into a SearchResponse so add_response still works.
2. Update the fixture() helper and related test assertions for the random route.
3. Smart and metadata keep using the wrapped SearchResponse format.
4. The AlbumWire defensive deserialization pattern already exists and can be mirrored for assets (vc1_73764c206007f30547943a5b9a4f47970c648d82fd665a6edfbce7d15ab8aaab, the review item, is the addition of array items — it just describes the new shape and requires no separate action beyond handling the array).

Shared symbol: SearchResponse / AssetResults / parse_search_response are shared across all search routes. The random-specific parser should be a private function that wraps the result into a SearchResponse without touching the public API of parse_search_response.

Now editing:

Applied diff to src/lib.rs >

Now add the post_random method to ImmichClient. I need to read the current file to get the exact line numbers for the insertion point.
```

Bob fixes random search and leaves the other searches alone.

Test the before and after

All 8 tests pass. Commit this revision and run verify:

```
Ran cd /home/nryn/work/sethu-demo-picker && git add src/lib.rs tests/search_picker.rs harness/test_
```

Now need to update the manifest's `patched_commit` to the new HEAD and re-freeze:

```
Ran cd /home/nryn/work/sethu-demo-picker && git rev-parse HEAD 2>&1
```

Applied diff to `verify-manifest.json`

Ran 4 commands

There's an incomplete run from the previous timed-out attempt. The `metadata-guard/patched-new` stage has a worktree but no results. Let me clean that up and re-run:

Ran 3 commands

The verify command from the frozen run left partial artifacts from the timed-out run. Let me clean those up:

Ran 2 commands

Clean working tree. Now run verify again:

```
Ran cd /home/nryn/work/sethu-demo-picker && sethu verify --manifest verify-manifest.json 2>&1
```

All three checks are **verified**. The regression shows the expected pattern:

- `random-picker original-old`: **pass** — baseline code works against old fixtures ✓
- `random-picker original-new`: **expected-red** — baseline code fails against new bare-array response ✓ (correct regression)
- `random-picker patched-new`: **pass** — patched code handles new bare-array response ✓
- Both guards pass all stages ✓

Commit the run artifacts:

```
Ran cd /home/nryn/work/sethu-demo-picker && git add runs/ && git -c user.email=migrator@example.co_
```

Old passes. New fails as expected. Repaired new passes.

27 changes accounted for

One needs a product decision: what should Next Page do without API paging?

Sethu

The bridge for API breaking changes. Powered by IBM Bob.

Narayan SS

Docs: noemaforge.github.io/sethu/

Code: github.com/noemaforge/sethu

Project links

Sethu docs noemaforge.github.io/sethu/

Sethu on GitHub github.com/noemaforge/sethu

Sethu on crates.io crates.io/crates/sethu

Vimanam on GitHub github.com/noemaforge/vimanam

Vimanam on crates.io crates.io/crates/vimanam